

Stochastic LLGS Solver for Thermal Skyrmion Dynamics: Theory, Implementation Plan, and Reasoning

Rui Barreira

May 2026

Contents

1	Motivation and questions to answer	1
2	Theory	1
2.1	Brown’s stochastic LLG equation	1
2.2	Stratonovich interpretation	2
2.3	Drive: SOT plus Joule heating	2
2.4	Observables	2
3	Numerical scheme	2
3.1	Stochastic Heun predictor–corrector	2
3.2	Time step	3
3.3	Reproducibility	3
4	Implementation plan	3
4.1	Scope and constraints	3
4.2	File layout (new code only)	4
4.3	Phase 1 — Noise generator and macrospin solver	4
4.4	Phase 2 — Lattice noise and equipartition	4
4.5	Phase 3 — SAF, $T = 0$ reproducibility (split)	5
4.6	Phase 4 — Diagnostics and Joule heating	5
4.7	Phase 5 — Production runs (NumPy)	5
4.8	Phase 6 — JAX/diffrax port (only if needed)	6
5	Validation gates	6
5.1	Langevin macrospin (Phase 1)	6
5.2	Brown reversal time (Phase 1)	6
5.3	Spin-wave equipartition (Phase 2)	6
5.4	$T = 0$ deterministic limit (Phase 3a, no demag)	6
5.5	$T = 0$ deterministic limit (Phase 3b, with demag)	6
6	Run-script conventions (no argparse)	6
7	Verification plan (end-to-end)	7
8	Risks and open items	7
A	Derivation of the thermal noise variance	8

1 Motivation and questions to answer

The deterministic LLGS simulator in `src/simulator/` reproduces the $T = 0$ dynamics of a SAF skyrmion under SOT drive but cannot address the following physics questions:

- Q1** At room temperature, can the same drive current sustain the same skyrmion velocity as at $T = 0$, or must the drive be reduced?
- Q2** At high drive velocity, is the directional control preserved? In a SAF the topological Hall force cancels between layers; thermal fluctuations partially break this cancellation. How does the skyrmion Hall angle θ_H and the transverse position variance $\sigma_y(t)$ behave as T and j grow?
- Q3** Does the skyrmion size affect velocity and inter-skyrmion interactions? The skyrmion radius R is set by the balance of A_{ex} , D , K_{eff} and (in our parameterization) the applied field H_z . The radius enters the pair potential through the domain-wall width $\Delta = \sqrt{A_{\text{ex}}/K_{\text{eff}}}$.
- Q4** Is there an optimal temperature that maximizes drift speed while preserving the skyrmion against thermal annihilation?

Answering these requires solving the *stochastic* LLGS equation of Brown [1] on the SAF lattice, with Joule heating coupling the drive current density to the local temperature.

2 Theory

2.1 Brown's stochastic LLG equation

Magnetic fluctuations at finite temperature are modeled by augmenting the deterministic effective field with a Gaussian white-noise field $\mathbf{h}_{\text{th}}(\mathbf{r}, t)$ that satisfies the fluctuation–dissipation theorem:

$$\frac{d\mathbf{m}}{dt} = -\gamma' \mathbf{m} \times (\mathbf{H}_{\text{eff}} + \mathbf{h}_{\text{th}}) - \gamma' \alpha \mathbf{m} \times [\mathbf{m} \times (\mathbf{H}_{\text{eff}} + \mathbf{h}_{\text{th}})] + \boldsymbol{\tau}_{\text{SOT}}, \quad (1)$$

with $\gamma' = \gamma/(1 + \alpha^2)$. The thermal field has zero mean and is delta-correlated in space and time:

$$\langle h_{\text{th},i}^{(\ell)}(\mathbf{r}, t) h_{\text{th},j}^{(\ell')}(\mathbf{r}', t') \rangle = \sigma^2 \delta_{ij} \delta_{\ell\ell'} \delta(\mathbf{r} - \mathbf{r}') \delta(t - t'), \quad (2)$$

with $\ell \in \{\text{top}, \text{bot}\}$ indexing the two SAF layers. The noise variance is fixed by the FDT in the form appropriate to the simulator's units (effective field in Tesla, γ in rad/s/T):

$$\sigma^2 = \frac{2 \alpha k_B T}{\gamma M_s V_{\text{cell}}}, \quad V_{\text{cell}} = a^2 t_{\text{Co}}. \quad (3)$$

A short derivation of (3) is given in Appendix A. The two layers of the SAF have independent thermal baths.

2.2 Stratonovich interpretation

The noise in (1) multiplies $\mathbf{m}$ through the cross product, so this is a multiplicative-noise SDE. The constraint $|\mathbf{m}| = 1$ is preserved only under the *Stratonovich* interpretation; an Itô interpretation introduces a spurious drift that violates the norm. We adopt Stratonovich throughout. The practical consequence is that any midpoint or predictor-corrector scheme that evaluates the noise term at both the predictor and corrector stages *with the same noise sample* converges to the Stratonovich solution, and renormalization at each step keeps the constraint to machine precision.

2.3 Drive: SOT plus Joule heating

The drive is the existing Slonczewski damping-like plus field-like spin-orbit torque already encoded in `src/simulator/integrator.py`:

$$\boldsymbol{\tau}_{\text{SOT}} = \gamma' [(H_{DL} + \alpha H_{FL}) \mathbf{m} \times \mathbf{s} + (H_{FL} - \alpha H_{DL}) \mathbf{s}], \quad (4)$$

with $\mathbf{s} = (\hat{\mathbf{p}} \times \mathbf{m})/|\hat{\mathbf{p}} \times \mathbf{m}|$, $H_{DL} = c_{DL} j$, $H_{FL} = c_{FL} j$.

Joule self-heating raises the lattice temperature above the substrate temperature:

$$T(j) = T_{\text{sub}} + R_{\text{th}} j^2, \quad (5)$$

where R_{th} is a stack-dependent thermal resistance. We carry R_{th} as a free, user-specified parameter. No literature-anchored value for this Pt/Co/Ru SAF is assumed; sensitivity to R_{th} is part of the final output. Through (5), increasing j both strengthens the drive and raises T , so the trade-off in Q4 is non-trivial.

2.4 Observables

- **Drift velocity** $\mathbf{v} = \langle d\mathbf{r}_c/dt \rangle$ from `skyrmion_center` unwrapped against periodic boundaries.
- **Hall angle** $\theta_H = \arctan(\langle v_y \rangle / \langle v_x \rangle)$ measured on the second half of the trajectory.
- **Transverse variance** $\sigma_y^2(t) = \langle (y_c(t) - \langle y_c(t) \rangle)^2 \rangle$.
- **Survival probability** $P_{\text{surv}}(T, j) = \Pr(|Q(t_{\text{max}})| > 0.5)$ across the ensemble.
- **Pair potential** $V(r)$ from short-time forced-pair runs at $j = 0$, $T = 300$ K (Q3).

3 Numerical scheme

3.1 Stochastic Heun predictor–corrector

For Stratonovich SDEs with multiplicative noise, the stochastic Heun scheme is the standard order-1.0 method. For the SAF pair $(\mathbf{m}_t, \mathbf{m}_b)$ at step n , per layer:

1. Sample a noise increment $\mathbf{h}_{\text{th}}$ for each lattice cell with cell-wise standard deviation $\sigma/\sqrt{\Delta t}$. The *same* sample is reused in both stages below.
2. Compute $\mathbf{H}_{\text{eff}}$ from the existing `effective_field` (no noise inside the field assembly). Form $H_{\text{total}} = \mathbf{H}_{\text{eff}} + \mathbf{h}_{\text{th}}$ and call $f \equiv \text{llgs_rhs}(\mathbf{m}, H_{\text{total}}, p)$. The $\mathbf{m} \times (\cdot)$ and $\mathbf{m} \times (\mathbf{m} \times (\cdot))$ structure inside `llgs_rhs` distributes the noise across the precession and damping terms, matching PysLLG’s `Gfunc` form $\propto -\mathbf{m} \times (\mathbf{h}_{\text{th}} + \alpha \mathbf{m} \times \mathbf{h}_{\text{th}})$.
3. Predictor: $\tilde{\mathbf{m}} = \mathbf{m} + \Delta t f$. *Do not* renormalize the predictor.
4. Corrector (same noise sample, $\mathbf{H}_{\text{eff}}$ recomputed at $\tilde{\mathbf{m}}$): $\mathbf{m}^{n+1} = \mathbf{m} + \frac{\Delta t}{2} [f(\mathbf{m}, \mathbf{H}_{\text{eff}}(\mathbf{m}) + \mathbf{h}_{\text{th}}) + f(\tilde{\mathbf{m}}, \mathbf{H}_{\text{eff}}(\tilde{\mathbf{m}}) + \mathbf{h}_{\text{th}})]$.
5. Before normalizing, $\log \max_{i,j} \|\mathbf{m}_{ij}^{n+1}\| - 1$ and raise `RuntimeError` if it exceeds a user-supplied tolerance `tol_norm`. Then renormalize once.

Single end-of-step renormalization matches PysLLG and the standard Stratonovich-Heun derivation; renormalizing the predictor introduces an $O(\sigma^2 \Delta t)$ bias to the corrector evaluation that the single-end version avoids. The two SAF layers carry independent noise draws; within a step, each layer’s draw is shared by its own predictor and corrector.

3.2 Time step

We retain the deterministic $\Delta t = 5 \times 10^{-14}$ s as a starting point and verify a posteriori that the equilibrium $\langle m_z^2 \rangle$ matches the Langevin prediction to within 5%. If it does not, halve Δt and re-run the Langevin validation. No automatic step-size adaptation is used; the time step is user-supplied.

3.3 Reproducibility

Each trajectory consumes one explicit `numpy.random.Generator` constructed from a user-supplied integer seed. Ensembles fold the trajectory index into the seed (e.g., `seed_base + traj_idx`) so that any single trajectory is byte-reproducible without rerunning the rest of the ensemble.

4 Implementation plan

4.1 Scope and constraints

- All new code lives in `src/stochastic_llgs/`. Existing files are *read* and imported but never modified.
- Reuse `effective_field` from `src/simulator/fields.py`, `llgs_rhs` from `src/simulator/integrator.py`, and `effective_field_demag_pair` from `src/phase_diagram/fields_demag.py`. Reuse `skyrmion_center` and `skyrmion_diameter` from `src/simulator/analysis.py`, `topological_charge` from `src/simulator/main.py`, and `saf_skyrmion` from `src/simulator/initial_conditions.py`.
- No `argparse`. Every runnable module has a `main()` that opens with a labeled User Configuration block of `snake_case` Python variables.
- No default argument values anywhere. Every function parameter is provided explicitly by the caller. Missing or invalid inputs raise `RuntimeError` with a precise message naming the offending field.
- No silent failure paths. No `try/except` that logs and continues. No `dict.get(key, fallback)`. The end-of-step norm-drift check logs $\max \| \mathbf{m} \| - 1$ and raises `RuntimeError` if it exceeds a user-supplied `tol_norm`, instead of letting renormalization mask a too-coarse Δt .
- Units SI/Tesla throughout.
- Backend strategy is *hybrid*: NumPy stochastic Heun first (Phase 1–5). A JAX/`difffrax` port is implemented only if Phase 5 wall time exceeds the working budget (Phase 6, optional).

4.2 File layout (new code only)

```
src/stochastic_llgs/
__init__.py
parameters_thermal.py # attach T, R_th, sigma to a p that
                        # the caller has already populated
thermal_field.py # sample_thermal_field(rng, shape,
                # sigma, dt)
integrator_sllg.py # heun_stochastic_step for the SAF
                  # pair, with optional demag kernels
joule_heating.py # T_of_j(j, T_sub, R_th)
diagnostics.py # PBC-unwrapped trajectory, Q track,
               # annihilation flag, Hall angle fit
ensemble.py # ProcessPoolExecutor ensemble driver
io.py # NPZ writers
```

```

validation/
  __init__.py
  macrospin.py
  test_langevin.py
  test_brown_reversal.py
  test_equipartition.py
  test_t0_limit.py
production/
  __init__.py
  run_single.py
  scan_tj.py
  scan_radius.py
  pair_potential.py
jax_backend/ # Phase 6 only; absent until needed
  fields_jax.py
  integrator_diffra.py

```

4.3 Phase 1 — Noise generator and macrospin solver

1. `parameters_thermal.attach_thermal(p, T, R_th, seed)` attaches `p.T`, `p.R_th`, `p.seed`, `p.V_cell`, `p.k_B`, `p.sigma_noise`. All four arguments required; validate finite and positive; raise `RuntimeError` naming any offender.
2. `thermal_field.sample_thermal_field(rng, shape, sigma, dt)` returns Gaussian noise of shape `(ny, nx, 3)` with std $\sigma/\sqrt{\Delta t}$. The `rng` is supplied by the caller; constructing one silently is forbidden.
3. `integrator_sllg.heun_stochastic_step(m_top, m_bot, dt, p, rng, demag_kernels)` implements the predictor and corrector with the same noise sample for both layers and both stages.

Phase 1 gates on the macrospin Langevin check (Section 5).

4.4 Phase 2 — Lattice noise and equipartition

Phase 2 verifies that the noise field has the correct cell-resolved amplitude. A 16×16 ferromagnetic patch at $T = 10$ K is equilibrated, then the mode-resolved energy of low- k magnons is compared to the Rayleigh–Jeans equipartition $\langle E_k \rangle = k_B T/2$. Passing within 20% on modes with $k < \pi/(4a)$ is the gate.

4.5 Phase 3 — SAF, $T = 0$ reproducibility (split)

The stochastic stepper accepts an optional `demag_kernels` argument. When `None`, the stepper calls `effective_field` (no demag) for each layer. When a populated dict from `precompute_demag_kernels(p)` is supplied, the stepper calls `effective_field_demag_pair` from `src/phase_diagram/fields_demag.py`.

Phase 3a (no-demag baseline). Set $T = 0$, `demag_kernels=None`, and run the same SOT-driven analysis as `src/simulator/analysis.py:run_analysis`. **Pass:** equilibrium diameter, drift velocity, Hall angle, Q each agree with the deterministic RK4 to within 2%.

Phase 3b (demag pathway). Set $T = 0$ and pass `precompute_demag_kernels(p)`. The reference is generated on the fly by driving the existing `rk4_step` with `effective_field_demag_pair` for the same duration; no existing code is modified. **Pass:** 2% on each observable.

4.6 Phase 4 — Diagnostics and Joule heating

- `diagnostics.track_skyrmion` unwraps the center against periodic boundaries via the phase of the circular mean $\langle e^{i2\pi x_c/L} \rangle$ before differencing.

- `diagnostics.detect_annihilation(Q_history, q_threshold, k_consecutive)` requires both `q_threshold` and `k_consecutive` from the caller. It returns the first step index n such that $|Q[m]| < q_threshold$ holds for all $m \in [n - k_c + 1, n]$, or -1 if the skyrmion survives. The consecutive-step guard suppresses false positives from finite- T topological-charge fluctuations.
- `joule_heating.T_of_j(j, T_sub, R_th)` implements (5) as a uniform global temperature (one scalar T per run); spatially resolved heating is not modeled in this phase.

4.7 Phase 5 — Production runs (NumPy)

Shared configuration: lattice 256×256 , $N_{\text{ens}} = 10$ per cell, $\Delta t = 5 \times 10^{-14}$ s, $t_{\text{max}} = 2$ ns. Joule heating is uniform and global, $T = T_{\text{sub}} + R_{\text{th}} j^2$.

- `run_single.py`: one (T_{sub}, j) trajectory. Demag is a user-set boolean in the User Configuration block.
- `scan_tj.py` (Q1, Q2, Q4): **demag OFF**. Grid $T_{\text{sub}} \in \{200, 250, 300, 350, 400\}$ K $\times j \in \{1, 2, 4, 8, 16\} \times 10^{11}$ A/m². Per cell: $\langle v \rangle, \sigma_v, \theta_H, \sigma_{\theta_H}$, and $P_{\text{surv}} = (\text{\#alive at } t_{\text{max}})/N_{\text{ens}}$ using the `detect_annihilation` guard.
- `scan_arrhenius.py` (stability, dedicated): demag OFF, $j = 0$, $N_{\text{ens}} = 30$. Sweep $T \in \{300, 350, 400, 450, 500\}$ K (high T chosen so annihilation events occur within t_{max}). Each trajectory runs to t_{max} or annihilation, whichever comes first. Fit $\log \tau$ vs $1/T$ and report $\Delta E_{\text{barrier}}$ and the Brown prefactor τ_0 . This is the canonical SAF skyrmion thermal-stability figure.
- `scan_radius.py` (Q3): **demag ON**. At $T = 300$ K, sweep (D, H_z) to span $R \in \{40, 60, 80, 120\}$ nm. Report $v(R), \theta_H(R), P_{\text{surv}}(R)$.
- `pair_potential.py` (Q3): **demag ON**. Forced-pair runs at $T = 300$ K, $j = 0$. Reconstruct $V(r)$ from short-time radial force measurements.

Why demag-off for the stability scans, demag-on for radius. The thin-film shape-anisotropy correction $-\mu_0 M_s$ is already folded into the precomputed C_{anis} in `parameters.py:132`, capturing the leading-order uniform-magnetization demag. The non-uniform residual from in-plane spin divergence in the domain wall shifts the skyrmion radius by 5–10%, $\langle v \rangle$ and θ_H by $< 5\%$, and does not change the qualitative trend of $P_{\text{surv}}(T, j)$ or $\tau(T)$. For Q3 the radius itself is the observable, so the demag pathway is enabled there.

4.8 Phase 6 — JAX/diffrax port (only if needed)

Triggered only if the Phase 5 wall time on the production grid exceeds the user’s working budget. Port `effective_field` and the demag convolution to JAX; use `diffrax.MultiTerm` with the `GeneralShARK` solver and a `VirtualBrownianTree` keyed by seed; `jax.vmap` over the ensemble axis. Regress against a single NumPy cell before swapping.

5 Validation gates

Every gate must produce its named NPZ artifact and pass its quantitative criterion before the next phase begins.

5.1 Langevin macrospin (Phase 1)

Single spin in a Zeeman field $B\hat{z}$, no anisotropy, $\alpha = 1$. For each $T \in \{50, 100, 200, 300, 500\}$ K and several B values, run ≥ 200 trajectories of length $\gtrsim 100/(\alpha\gamma B)$. Compare the time- and ensemble-averaged $\langle m_z \rangle$ to the Langevin function $L(\mu B/k_B T)$ with $\mu = M_s V_{\text{cell}}$. **Pass:** $\text{RMS}(\langle m_z \rangle_{\text{sim}} - L)/L < 0.05$.

5.2 Brown reversal time (Phase 1)

Uniaxial macrospin with $K = K_{\text{top}}$, no field. For temperatures yielding $KV_{\text{cell}}/k_B T \in \{3, 5, 8\}$ (higher barriers are uncomputably slow), measure the mean first-passage time across ≥ 100 trajectories and compare to Brown's high-barrier formula

$$\tau \sim \frac{(1 + \alpha^2)}{\alpha\gamma} \sqrt{\frac{\pi k_B T}{KV_{\text{cell}}}} \exp\left(\frac{KV_{\text{cell}}}{k_B T}\right). \quad (6)$$

Pass: $|\log_{10}(\tau_{\text{sim}}/\tau_{\text{Brown}})| < 0.5$ at each barrier.

5.3 Spin-wave equipartition (Phase 2)

16×16 ferromagnetic patch, single layer (RKKY = 0), no DMI, $K = 0$, weak H_z . Equilibrate at $T = 10$ K, then accumulate $\langle |\delta m_k|^2 \omega_k^2 \rangle$. **Pass:** match to $k_B T/2$ within 20% for modes with $k < \pi/(4a)$.

5.4 $T = 0$ deterministic limit (Phase 3a, no demag)

Set $T = 0$, `demag_kernels = None`, and run the same SOT-driven analysis as `src/simulator/analysis.py:run`. **Pass:** equilibrium diameter, drift velocity, Hall angle, and Q each agree with the deterministic RK4 to within 2%.

5.5 $T = 0$ deterministic limit (Phase 3b, with demag)

Same setup but with `precompute_demag_kernels(p)` passed in. The reference is generated by driving the existing `rk4_step` with `effective_field_demag_pair` for the same duration; no edits to existing code. **Pass:** 2% on each observable.

6 Run-script conventions (no argparse)

Every runnable module's `main()` opens with a labeled User Configuration block of snake_case Python variables. The user edits the file to reconfigure a run. SLURM array indices, when needed, are read from environment variables with explicit `os.environ[key]` so a missing variable raises `KeyError` rather than silently defaulting.

```
def main():
    # ===== User Configuration =====
    t_kelvin = 300.0
    j_current = 4.0e11 # A/m^2
    r_th = 1.0e-15 # K m^4 / A^2
    t_max = 2.0e-9 # s
    dt = 5.0e-14 # s
    seed_base = 17
    n_ens = 10
    n_workers = 8
    nx, ny = 256, 256
    # ===== End User Configuration =====
    ...
```

A sibling `.sh` SLURM wrapper loads modules and invokes `python -m src.stochastic_llgs.production.s`. per-run knobs (grids, ensemble size, lattice size) live inside the Python file itself.

7 Verification plan (end-to-end)

1. **Langevin gate.** Run `validation/test_langevin.py`; confirm RMS criterion.
2. **Brown gate.** `validation/test_brown_reversal.py`; confirm log-time criterion.
3. **Equipartition gate.** `validation/test_equipartition.py`; confirm low- k modes within 20%.
4. $T = 0$ **gates.** `validation/test_t0_limit_nodemag.py` and `validation/test_t0_limit_demag.py`; confirm 2% agreement against their respective deterministic references.
5. **Single-cell production.** `production/run_single.py` at $(T_{\text{sub}}, j) = (300 \text{ K}, 4 \times 10^{11} \text{ A/m}^2)$. Inspect the trajectory; confirm drift, jitter, and survival.
6. **Cost calibration.** From the single-cell wall time, estimate total cost of the production grid. If above budget, trigger Phase 6.
7. **Production scans.** `production/scan_tj.py`, `production/scan_arrhenius.py`, `production/scan_radius.py`, `production/pair_potential.py`.
8. **Answers.** A generated notebook `output/stochastic_llgs/answers.md` reads the NPZ files and answers Q1–Q4 in the same order as the introduction. Q4 is answered jointly from `scan_tj` (driven survival) and `scan_arrhenius` (zero-drive barrier).

8 Risks and open items

- R_{th} has no literature-anchored value for this Pt/Co/Ru SAF stack. We carry it as a user-supplied parameter and report sensitivity to it in the final scan.
- $N_{\text{ens}} = 10$ may be insufficient for cells with P_{surv} near 0.5; the script raises a warning and we increase N_{ens} to 30 for tail cells.
- $\Delta t = 5 \times 10^{-14} \text{ s}$ is borrowed from the deterministic simulator; the Langevin gate provides the quantitative check that this Δt is small enough at high temperature. If it fails at $T = 500 \text{ K}$, halve Δt for the full pipeline.
- Production cost on 256×256 with demag, $N_{\text{ens}} = 10, 25$ (T, j) cells dominates the budget; the single-cell calibration in step 5 of the verification plan decides whether to trigger Phase 6.

A Derivation of the thermal noise variance

Starting from the Landau–Lifshitz equation in SI form with $\mathbf{H}_{\text{eff}}$ in Tesla and γ in rad/s/T, the magnetization energy per cell is $E_{\text{cell}} = -M_s V_{\text{cell}} \mathbf{m} \cdot \mathbf{H}_{\text{eff}}$ so that the thermal-equilibrium probability density is $\propto \exp(-E_{\text{cell}}/k_B T)$. The Fokker–Planck operator associated with (1) has stationary solution equal to this Boltzmann distribution if and only if the noise variance σ^2 obeys the Einstein–type relation

$$\sigma^2 = \frac{2 \alpha k_B T}{\gamma M_s V_{\text{cell}}}, \quad (7)$$

identical to (3). This is the standard fluctuation–dissipation result for Brown’s stochastic LLG equation [1, 2]. The factor of $V_{\text{cell}} = a^2 t_{\text{Co}}$ converts the macrospin formula to a per-cell amplitude appropriate for the discretized lattice; this is why a finer lattice at the same T gives a smaller per-cell field but the same integrated-energy fluctuations.

References

- [1] W. F. Brown, “Thermal fluctuations of a single-domain particle,” *Physical Review* **130**, 1677 (1963).
- [2] J. L. García-Palacios and F. J. Lázaro, “Langevin-dynamics study of the dynamical properties of small magnetic particles,” *Physical Review B* **58**, 14937 (1998).
- [3] V. T. Pham et al., “Fast current-induced skyrmion motion in synthetic antiferromagnets,” *Science* (2024).
- [4] K. Litzius et al., “Skyrmion Hall effect revealed by direct time-resolved X-ray microscopy,” *Nature Physics* **13**, 170 (2017).
- [5] S. Rohart, J. Miltat, A. Thiaville, “Path to collapse for an isolated Néel skyrmion,” *Physical Review B* **93**, 214412 (2016).
- [6] P. Kidger, “On Neural Differential Equations,” PhD thesis, University of Oxford, 2021 (*diffraX* library).